

SILAB[®]

Version 7.1
DPUSharedMemory

DPUSharedMemory

Allgemeine DPU zur Kommunikation über Shared Memory.

Die Anzahl und Art der Variablen, die übertragen werden sollen, kann im DPU-Konfigurationsskript festgelegt werden.

Statische Parameter:

<i>SHMName</i>	Name der Shared-Memory-Sektion.
<i>SHMMode</i>	Modus zum Öffnen der DPU. Möglich sind: 1 Immer öffnen, nie erstellen. (Führt zu Fehler, falls der andere Prozess die Shared Memory noch nicht erstellt hat.) 2 Immer erstellen, nie öffnen. (Führt zu Fehler, falls die Shared Memory bereits existiert.) 3 Öffnen oder erstellen, falls noch nicht vorhanden.
<i>SHMMaxLock</i>	Maximalzahl von Versuchen zum Sperren der Shared Memory, falls eine Variable vom Typ <i>lock</i> definiert ist. (s.u.)
<i>SendDefinition</i>	Definition der Werte, die in die Shared Memory geschrieben werden sollen.
<i>ReceiveDefinition</i>	Werte, die aus der Shared Memory ausgelesen werden sollen.

Ein- und Ausgänge:

(beliebig) Können im Skript definiert werden.

Funktionsweise:

Die *DPUSharedMemory* öffnet eine Shared-Memory-Sektion und kommuniziert über diese mit einem anderen Prozess, der auf demselben Rechner läuft.

Über die Parameter ‚ReceiveDefinition‘ bzw. ‚SendDefinition‘ das Skript werden Bereiche der Shared Memory definiert, die von der DPU ausgelesen bzw. beschrieben werden sollen:

```
<SHMDefinition> =
(
    (,<VariablenName> ,<Datentyp> [,<Defaultwert> , <Multiplikator> , <Offset>] )
);
```

Die in der Definition angegebenen Variablennamen stellt die *DPUSharedMemory* dann als Ein- bzw. Ausgänge zur Verfügung. Eine Übersicht der unterstützten Datentypen bietet die folgende Tabelle:

Datentyp	Datenlänge (Byte)	Min	Max
byte	1	-128	+127
ubyte	1	0	+255
word	2	-32768	+32767
uword	2	0	+65535
long	4	-2147483648	+2147483647
ulong	4	0	+4294967295

longlong	8	$-9 \cdot 10^{18} *$	$9 \cdot 10^{18} *$
ulonglong	8	0	$4 \cdot 10^{19} *$
float	4	$3.4 \cdot 10^{+38}$ (7 Stellen)	
double	8	$1.7 \cdot 10^{+308}$ (15 Stellen)	
string(n)	n	Zeichenkette mit n Ascii-Zeichen	
pad(n)	n	Speicherbereich der ignoriert werden soll	
status	4	Zustand von SILAB (s.u.)	
lock	4	Variable zur Synchronisation des Zugriffs (s.u.)	
event_hold(n)	4	Ereignisvariable (s.u.)	
const	4	Konstante (s.u.)	
simtime	4	Zählt Millisekunden seit dem Start der Simulation (nur als Ausgang)	
silabtime	4	Zählt Millisekunden seit dem Start von SILAB (nur als Ausgang)	

Der erste Eintrag wird am Anfang der Shared-Memory-Sektion abgelegt.

Sowohl die *SendDefinition* als auch die *ReceiveDefinition* beziehen sich auf dieselbe Shared-Memory-Sektion. Die Struktur wird jeweils von vorne an durchlaufen. Falls ein Speicherbereich nicht gelesen bzw. beschrieben werden soll, muss hier jeweils eine pad-Variable mit der entsprechenden Länge eingefügt werden. Der Name dieser pad-Variable wird ignoriert, es wird kein zugehöriger DPU-Eingang bzw. -Ausgang angelegt.

Beispiel:

Die Shared Memory soll eine Datenstruktur enthalten, die in C wie folgt definiert sein könnte:

```
typedef struct {
    long Lock; /* Wert zur Synchronisation (s.u.) */
    double Data1; /* soll von SILAB ausgelesen werden */
    double Data2; /* soll in SILAB nicht verwendet werden */
    double Data3; /* soll von SILAB aus beschrieben werden */
} SHMStruct;
```

Dann könnte die *DPUSharedMemory* wie folgt instanziiert werden, um die Kommunikation durchzuführen:

```
DPUSharedMemory Memory
{
    Computer = {DATA};
    Index = 99;
    SHMMode = 3;
    SHMName = "SHMStruct";

    SendDefinition = (
        (dummy1, pad(20)), # Lock + zwei Double-Werte (4+2*8=20 bytes) übergehen
        (Data3, double) # dritten Double-Wert beschreiben
    );
    ReceiveDefinition = (
        (lock, lock), # Zugriffe auf Speicher synchronisieren (s.u.)
        (Data1, double) # ersten Double-Wert auslesen
    );
}
```

* In SILAB ist die Genauigkeit bei der Verwendung von long-long-Werten auf 15 Stellen beschränkt, d.h. Ganzzahlen größer als 10^{15} können nicht exakt dargestellt werden.

```
);
};
```

Nach dem Laden der Konfiguration wird die DPU ‚Memory‘ in SILAB den Ausgang `Data3` und den Eingang `Data1` besitzen, worüber die Kommunikation stattfinden kann.

Der Datentyp *status*:

Wenn eine Variable vom Typ *status* in der *SendDefinition* aufgeführt wird, wird dafür kein DPU-Eingang angelegt, sondern stattdessen wird die Variable stets mit dem aktuellen Zustand der Simulation gefüllt:

Status	Bezeichnung	Beschreibung
0	NotRunning	SILAB läuft derzeit nicht (oder ist noch nicht zur Kommunikation bereit).
1	Starting	Die Simulation wird derzeit gestartet.
2	Running	Die Simulation läuft derzeit.
3	Paused	Die Simulation ist derzeit angehalten.
4	Stopping	Die Simulation wird derzeit beendet.

Der Datentyp *lock*:

Durch den Zugriff zweier nicht synchronisierter Prozesse auf denselben Speicherbereich (die Shared Memory) kann es zu Konflikten beim Zugriff auf den Speicher kommen. Dies gilt insbesondere, wenn die Prozesse auf Multi-Core-Prozessoren ausgeführt werden, d.h. möglicherweise gleichzeitig laufen.

Ohne Synchronisation könnten **inkonsistente** Werte aus der Shared Memory ausgelesen werden, d.h. die ausgelesenen Werte passen nicht zusammen (ein Teil der Werte gehört zum alten Wertesatz, ein anderer Teil bereits zum neuen), oder es können **korrupte** Werte ausgelesen werden (ein Teil der Variable wurde bereits verändert, ein anderer Teil noch nicht).

Zur Synchronisation des Zugriffs kann SILAB ein *Spinlock* verwenden. Bei diesem sehr einfachen und schnellen Synchronisationsmechanismus versucht ein Prozess wiederholt, den Wert einer Variablen von 0 auf Nicht-0 zu verändern („test-and-set“-Operation).

In C kann die *Lock*-Operation wie folgt implementiert werden:

```
bool Lock(long* p_Lock, unsigned int ui_NTries) {
    for(int i = 0; i < ui_NTries; i++)
        if(!InterlockedExchange(p_Lock, 1))
            return true; // Lock erhalten
    return false; // Lock nicht erhalten
}
```

Der *Lock*-Funktion wird als Parameter ein Zeiger auf den Speicherbereich übergeben, der für die Synchronisation verwendet werden soll (dies ist die zugehörige 4 byte lange Variable in der Shared-Memory-Sektion).

Falls `Lock()` `true` zurückgibt, konnte der Prozess das *Spinlock* erhalten und es darf auf den Rest der Shared-Memory-Sektion zugegriffen werden.

Falls `Lock()` `false` zurückgibt, konnte in der angegebenen Anzahl von Versuchen (die jeweils nur wenige Prozessortakte dauern!) das *Spinlock* nicht erhalten werden, und in diesem Zyklus sollte kein Zugriff auf die Shared-Memory-Sektion erfolgen.

Nach dem synchronisierten Lesen und Schreiben der Werte muss der Prozess dann *Unlock* aufrufen, um den anderen Prozessen wieder den Zugriff zu gestatten:

```
void Unlock(long* p_Lock)
{
    InterlockedExchange(p_Lock, 0);
}
```

Die *DPUSharedMemory* synchronisiert alle Speicherzugriffe mit Hilfe eines *Spinlocks*, wenn eine *lock*-Variable in der *SendDefinition* oder in der *ReceiveDefinition* aufgeführt ist. (‚Send’ oder ‚Receive’ spielt hier keine Rolle; für *lock*-Variablen wird in keinem Fall ein DPU-Eingang bzw. Ausgang angelegt.) Für die *lock*-Variable werden in der Shared-Memory-Sektion 4 byte reserviert. (Dies muss ggf. bei der Definition der entgegengesetzten Kommunikationsrichtung berücksichtigt werden.)

Der Wert für `ui_NTries` kann mit dem DPU-Parameter *SHMMaxLock* eingestellt werden.

Der Datentyp *event_hold*:

Der Datentyp *event_hold(hold_time)* implementiert ein Protokoll zur sicheren Übertragung von Ereignissen.

Wenn eine Variable vom Typ *event_hold* in der *SendDefinition* aufgeführt wird, wird nur bei Änderung des zugehörigen DPU-Eingangs die Shared Memory – Variable beschrieben, und zwar dann wenn der neue DPU-Eingangswert ungleich 0 ist.

Wird eine *event_hold*-Variable in der *ReceiveDefinition* aufgeführt, dann wird die Shared-Memory-Variable auf Änderungen überwacht. Sobald ein Wert ungleich 0 erkannt wird, wird der DPU-Ausgang auf diesen neuen Wert gesetzt, und die Shared-Memory-Variable sofort wieder auf 0 zurückgesetzt. Nach Ablauf von *hold_time* ms wird dann der DPU-Ausgang auf 0 zurückgesetzt. Durch die Wartezeit *hold_time* wird sichergestellt, dass nachfolgende (möglicherweise langsamer getaktete) DPUs das Ereignis sicher empfangen können.

Der Datentyp *const*:

Ein *const*-Wert in der *SendDefinition* führt dazu, dass die Shared-Memory an dieser Stelle jederzeit mit dem Defaultwert des *const*-Wertes beschrieben wird.

Ein *const*-Wert in der *ReceiveDefinition* führt dazu, dass der Shared-Memory-Wert mit dem Defaultwert des *const*-Wertes verglichen wird. Falls diese nicht übereinstimmen, wird eine Warnmeldung ausgegeben.

Dieser Datentyp kann zur Überprüfung der korrekten Kommunikation verwendet werden, indem jeweils am Anfang und am Ende der Shared-Memory-Sektion ein *const*-Wert eingefügt wird. Somit wird sichergestellt, dass alle Kommunikationspartner eine kompatible Definition der Shared Memory verwenden.